

Accounting Firm Management & Financial Record Database

Kurian Zacharia Vadakara
UBIT: kurianva

Krish Puwar
UBIT: krishpuw

David Pang
UBIT: dpang3

I. PROJECT DETAILS

Project Title: Accounting Firm Management & Financial Record Database

Team Name: KQD

Team Members:

- 1) Kurian Zacharia Vadakara, kurianva
- 2) Krish Puwar, krishpuw
- 3) David Pang, dpang3

II. SQL QUERIES AND TESTING

A. Insert, Update, Delete Queries

Insert Queries:

```
1 INSERT INTO departments (department_name,  
2 manager_id)  
VALUES ('IT', NULL);
```

Listing 1. Insert Expense Record

	department_id [PK] integer	department_name text	manager_id integer
1	1	Tax	[null]
2	4	HR	[null]
3	2	Advisory	[null]
4	3	Audit	[null]

Fig. 1. Before Insert Query Execution - No IT Department

	department_id [PK] integer	department_name text	manager_id integer
1	1	Tax	[null]
2	2	Advisory	[null]
3	3	Audit	[null]
4	4	HR	[null]
5	5	IT	[null]

Fig. 2. After Insert Query Execution - IT department added

```
1 INSERT INTO Expenses (expense_id, expense_name,  
2 amount, department_id, user_id)  
VALUES (350, 'Office Supplies', 4540.65, 1,  
659);
```

Listing 2. Insert Expense Record

	expense_id [PK] integer	expense_name text	amount numeric (15,2)	department_id integer	user_id integer
339	340	Client Lunch	2577.29	1	281
340	341	Client Lunch	2592.01	4	487
341	342	Training Workshop	2301.58	4	589
342	343	Taxi Fare	1657.60	3	936
343	344	Conference Fee	1040.77	5	923
344	345	Training Workshop	4087.14	3	121
345	346	Cloud Subscription	2719.78	4	981
346	347	Office Supplies	3359.65	2	219
347	348	Training Workshop	2523.15	3	715
348	349	Client Lunch	3707.00	4	627

Fig. 3. Before Insert Query Execution - Showing expenses 340-349

	expense_id [PK] integer	expense_name text	amount numeric (15,2)	department_id integer	user_id integer
338	339	Cloud Subscription	948.34	4	59
339	340	Client Lunch	2577.29	1	281
340	341	Client Lunch	2592.01	4	487
341	342	Training Workshop	2301.58	4	589
342	343	Taxi Fare	1657.60	3	936
343	344	Conference Fee	1040.77	5	923
344	345	Training Workshop	4087.14	3	121
345	346	Cloud Subscription	2719.78	4	981
346	347	Office Supplies	3359.65	2	219
347	348	Training Workshop	2523.15	3	715
348	349	Client Lunch	3707.00	4	627
349	350	Office Supplies	4540.65	1	659

Fig. 4. After Insert Query Execution - expense_id 350 added

Update Queries:

```
1 UPDATE Departments  
2 SET manager_id = 1000  
3 WHERE department_id = 5;
```

Listing 3. Update Department Manager

	department_id [PK] integer	department_name text	manager_id integer
1	1	Tax	[null]
2	2	Advisory	[null]
3	3	Audit	[null]
4	4	HR	[null]
5	5	IT	[null]

Fig. 5. Before Update Query Execution - manager_id is NULL for department_id 5

	department_id [PK] integer	department_name text	manager_id integer
1	1	Tax	[null]
2	2	Advisory	[null]
3	3	Audit	[null]
4	4	HR	[null]
5	5	IT	1000

Fig. 6. After Update Query Execution - manager_id set to 1000 for department_id 5

```

1 UPDATE departments
2 SET manager_id = 3
3 WHERE department_id = 2;
4
5 UPDATE departments
6 SET manager_id = 4
7 WHERE department_id = 3;

```

Listing 4. Update Department Manager

	department_id [PK] integer	department_name text	manager_id integer
1	1	Tax	[null]
2	2	Advisory	[null]
3	3	Audit	[null]
4	4	HR	[null]
5	5	IT	1000

Fig. 7. Before Update Query Execution - manager_id is NULL for department_id 2 and 3

	department_id [PK] integer	department_name text	manager_id integer
1	1	Tax	[null]
2	4	HR	[null]
3	5	IT	1000
4	2	Advisory	3
5	3	Audit	4

Fig. 8. After Update Query Execution - manager_id set to 3 for department_id 2 and 4 for department 3

Delete Queries:

```

1 DELETE FROM Users
2 WHERE user_id = 1003;

```

Listing 5. Delete User Record

	user_id [PK] integer	first_name text	last_name text	email text	department_id integer	role text
1	1000	LeBron	James	lebron@buffalo.edu	6	Director of IT
2	1001	Kurian	Vadakara	kurianva@buffalo.edu	6	Associate
3	1002	Krish	Puwar	krishpuw@buffalo.edu	6	Associate
4	1003	Josh	Allen	joshallen1@buffalo.edu	6	Associate
5	1004	Lamar	Jackson	lamarjack1@buffalo.edu	6	Associate

Fig. 9. Before Delete Query Execution - Showing user_id 1003 (Josh Allen)

	user_id [PK] integer	first_name text	last_name text	email text	department_id integer	role text
1	1000	LeBron	James	lebron@buffalo.edu	6	Director of IT
2	1001	Kurian	Vadakara	kurianva@buffalo.edu	6	Associate
3	1002	Krish	Puwar	krishpuw@buffalo.edu	6	Associate
4	1004	Lamar	Jackson	lamarjack1@buffalo.edu	6	Associate

Fig. 10. After Delete Query Execution - user_id 1003 removed

```

1 DELETE FROM Expenses
2 WHERE user_id = 837 and expense_name = 'Taxi
   Fare';

```

Listing 6. Delete Expense Record

	expense_id [PK] integer	expense_name text	amount numeric (15,2)	department_id integer	user_id integer
1	1	Office Supplies	1892.50	5	284
2	2	Taxi Fare	3828.99	5	837
3	3	Taxi Fare	294.20	4	860
4	4	Training Workshop	1847.04	1	758
5	5	Conference Fee	2821.36	1	795
6	6	Conference Fee	477.50	4	40
7	7	Training Workshop	646.64	1	713
8	8	Cloud Subscription	463.85	5	33
9	9	Conference Fee	4667.64	1	506
10	10	Taxi Fare	4784.17	5	779
11	11	Training Workshop	64.90	4	938
12	12	Cloud Subscription	4799.24	1	733

Fig. 11. Before Delete Query Execution - Showing user_id 837's Taxi Fare Expense

	expense_id [PK] integer	expense_name text	amount numeric (15,2)	department_id integer	user_id integer
1	1	Office Supplies	1892.50	5	284
2	3	Taxi Fare	294.20	4	860
3	4	Training Workshop	1847.04	1	758
4	5	Conference Fee	2821.36	1	795
5	6	Conference Fee	477.50	4	40
6	7	Training Workshop	646.64	1	713
7	8	Cloud Subscription	463.85	5	33
8	9	Conference Fee	4667.64	1	506
9	10	Taxi Fare	4784.17	5	779
10	11	Training Workshop	64.90	4	938
11	12	Cloud Subscription	4799.24	1	733
12	13	Office Supplies	4540.65	1	659

Fig. 12. After Delete Query Execution - Taxi Fare Expense for user_id 837 is deleted

B. SELECT Queries

SELECT Query 1: GROUP BY with Aggregation

```

1 SELECT d.department_name, SUM(e.amount) AS
   total_expenses
2 FROM Departments d
3 JOIN Expenses e ON d.department_id =
   e.department_id
4 GROUP BY d.department_name
5 ORDER BY total_expenses DESC;

```

Listing 7. Department-wise Expense Summary

	department_name text	total_expenses numeric
1	Advisory	210467.16
2	HR	177850.47
3	Tax	174173.91
4	IT	164440.63
5	Audit	132909.48

Fig. 13. Result of SELECT Query 1 - Department Expenses with GROUP BY

SELECT Query 2: ORDER BY with Filtering

```
1 SELECT * FROM Expenses
2 ORDER BY amount DESC;
```

Listing 8. Expenses are listed in Decreasing Order

	expense_id [PK] integer	expense_name text	amount numeric (15,2)	department_id integer	user_id integer
1	134	Client Lunch	4994.34	2	537
2	307	Training Workshop	4965.43	1	480
3	79	Conference Fee	4901.84	2	138
4	60	Training Workshop	4897.58	2	187
5	63	Conference Fee	4894.86	2	325
6	97	Cloud Subscription	4894.36	1	116
7	44	Taxi Fare	4881.49	4	500
8	263	Training Workshop	4853.79	5	301
9	106	Conference Fee	4844.35	5	613
10	14	Taxi Fare	4843.74	5	501

Fig. 14. Result of SELECT Query 2 - Expenses Sorted in Decreasing Order

```
1 SELECT * FROM assets
2 ORDER BY purchase_date ASC;
```

Listing 9. Assets listed in Ascending Order based on Purchase Date

	asset_id [PK] integer	asset_name text	asset_type text	purchase_date date	value numeric (15,2)	department_id integer
1	117	Would Max	Server	2022-12-09	34013.67	1
2	46	Report Max	Desktop	2023-01-08	10176.54	4
3	102	Network Plus	Desktop	2023-01-10	47573.65	1
4	68	Suffer Plus	Laptop	2023-01-10	24307.15	3
5	22	Way Pro	Printer	2023-02-07	44863.23	2
6	127	Write Max	Vehicle	2023-02-10	27495.72	3
7	43	Admit Pro	Furniture	2023-02-17	7630.05	1
8	76	Door Max	Printer	2023-02-17	5682.04	3
9	15	Until Pro	Desktop	2023-03-10	2920.28	4
10	74	Set Pro	Desktop	2023-03-15	10218.28	2
11	41	Owner Pro	Printer	2023-04-22	34054.02	5
12	114	Seven Pro	Desktop	2023-04-23	27695.76	3

Fig. 15. Result of SELECT Query 3 - Assets Sorted in Ascending Order based on Purchase Date

SELECT Query 3: Subquery Filtering

```
1 SELECT *
2 FROM expenses e
3 JOIN users u
4 ON e.user_id = u.user_id
5 WHERE e.amount > (SELECT AVG(amount) FROM
6   expenses);
```

Listing 10. Expenses that above average expense price

	expense_id integer	expense_name text	amount numeric (15,2)	department_id integer	user_id integer
1	229	Training Workshop	3041.50	5	3
2	128	Conference Fee	4520.58	2	5
3	304	Training Workshop	2705.35	3	8
4	248	Taxi Fare	2561.23	2	8
5	257	Cloud Subscription	3156.85	4	14
6	132	Office Supplies	3118.26	1	24
7	217	Office Supplies	4220.38	2	28
8	99	Training Workshop	2688.22	4	31
9	191	Taxi Fare	4541.36	3	32

Fig. 16. Result of SELECT Query 3 - Assets Filtered by Subquery

C. Stored Procedures / Functions

Stored Procedure:

```
1 create or replace procedure add_asset(p_name
2   varchar, p_type varchar, p_date date,
3   p_value numeric, p_department int)
4 language plpgsql
5 as $$
6 begin
7   insert into assets(asset_name, asset_type,
8     purchase_date, value, department_id)
9   values (p_name, p_type, p_date, p_value,
10    p_department);
11 commit;
12 end;
13 $$;
14
15 call add_asset('building', 'property',
16   '2025-02-16 09:15:44', 4000000, 4)
```

Listing 11. Add Asset Procedure

	asset_id [PK] integer	asset_name text	asset_type text	purchase_date date	value numeric (15,2)	department_id integer
1	150	building	property	2025-02-16	4000000.00	4

Fig. 17. add_asset Procedure adds building into Asset Table

Function:

```
1 create function dept_user_count(dept_name
2   VARCHAR)
3 returns INTEGER
4 language plpgsql as $$
5 DECLARE d_count INTEGER;
6 BEGIN
7   SELECT COUNT(*)
8   INTO d_count
9   FROM users u
10  JOIN departments d ON u.department_id =
11    d.department_id
12  WHERE d.department_name = dept_name;
13 RETURN d_count;
14 END;
15 $$;
16
17 SELECT dept_user_count('Tax');
```

Listing 12. Department User Count Function

	dept_user_count
	integer
1	185

Fig. 18. Count the number of users in Tax Department

III. QUERY OPTIMIZATION & INDEXING

A. Problematic Queries

Query 1:

```
1 SELECT *
2 FROM Expenses
3 WHERE user_id = 487;
```

Listing 13. Slow Query: User Expenses

The user_id in the expense table is the foreign key of the table for relations, but it's not indexed and is unordered.

Cost: 0 - 7.36

Optimized Version:

```
1 CREATE INDEX idx_expenses_user ON
  Expenses(user_id);
```

Listing 14. Index on Expenses.user_id

To improve performance, a secondary dense index allows for faster aggregation of data on user_id

Query 2:

```
1 SELECT u.first_name, u.last_name,
2        d.department_name
3 FROM Users u
4 JOIN Departments d ON u.department_id =
5        d.department_id;
```

Listing 15. Slow Query: Users in Department

The problem here is that the department_id in the Users table is used frequently for joins to solve query problems, as it is the foreign key and also one of the main Primary keys in the relation. When the data for the users grows to over 5000, scanning the users table becomes expensive and time consuming.

Cost: 0.16 - 48.64

Optimized Version:

```
1 CREATE INDEX idx_users_department ON
  Users(department_id);
```

Listing 16. Index on Users.department_id

This works because it is a secondary dense index on a foreign key and helps reduce I/O time, and also using the join

function no longer requires scanning every tuple in both tables.

Query 3:

```
1 SELECT department_id, COUNT(*), SUM(value)
2 FROM Assets
3 GROUP BY department_id;
```

Listing 17. Slow Query: Recent Transactions

The problem with this query is that the database must examine every row in the Assets column, even if the user only needs assets from a certain department. Since Department_id is not indexed and it is not the primary key, therefore, this requires a secondary dense index.

Cost: 4.61 - 4.67

Optimized Version:

```
1 CREATE INDEX idx_assets_department
2 ON Assets (department_id);
```

Listing 18. Index on department_id

Since the file is unordered by department_id, it is a secondary dense index. With this method, fewer blocks are read, and it aggregates faster.

IV. DATABASE IMPLEMENTATION & SQL SCRIPTS

Data Source: All tables were created based on the Phase 1 schema specifications.

SQL:

- create.sql — contains SQL statements to create all tables with primary and foreign keys.
- load.sql — contains SQL statements to bulk load data into all tables from CSV or tab-separated files.
- query.sql — contains all the queries required for the project.

Tools: SQL as the database language and PostgreSQL as the database management system
Directory Contents: The project folder includes KurianZachariaVadakara_KrishPuwar_DavidPang_phase2.pdf, phase2_recording.mov, create.sql, query.sql, load.sql, all data files (.csv), and a readme.txt describing how tables were created and data were imported.

V. TEAM CONTRIBUTIONS

All team members (Kurian, Krish, David) contributed equally to creating, loading, and performing operations on the database.

VI. BONUS

- a) While we didn't write SQL queries directly in Tableau to extract the data, we exported our database tables as .csv files using PgAdmin/Postico. These .csv files were then imported into Tableau as text data sources to create the visualizations (see Fig. 19).

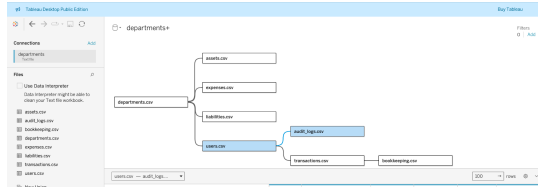


Fig. 19. Data model showing the linkage of all CSV tables through their key relationships, ensuring accurate dashboard reporting.

- b) **Dashboard Link:** https://public.tableau.com/app/profile/krish.puwar/viz/Book1_17641109643290/Dashboard1#1